THE CHIPS TO SYSTEMS CONFERENCE
63
SPONSORED BY:
CEDA
IEEE Council on Electronic Design Automation
sigda
acm

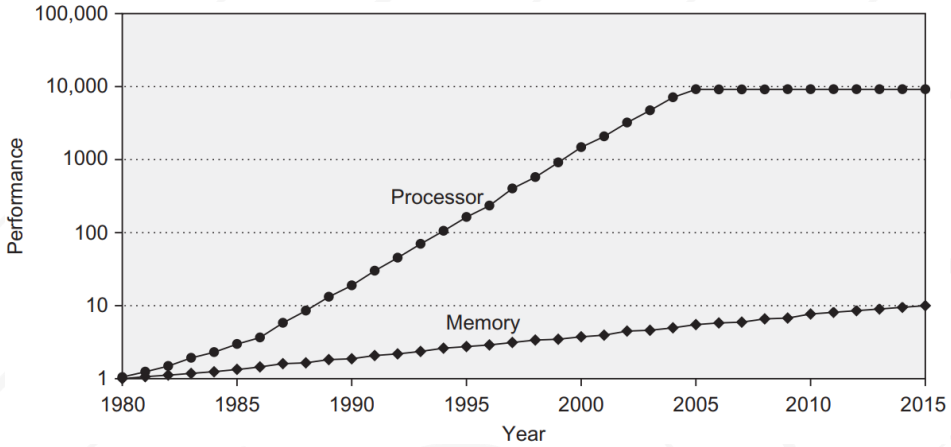
Prism: Reducing Arithmetic for High-Recall Approximate Nearest Neighbor Search on Processing-in-Memory

Weihan Kong[1], Shengan Zheng[1*], Yingxue Zhou[1], Yifan Hua[2], Yuheng Wen[1], Cong Zhou[1], Guifeng Wang[1], Linpeng Huang[1*]

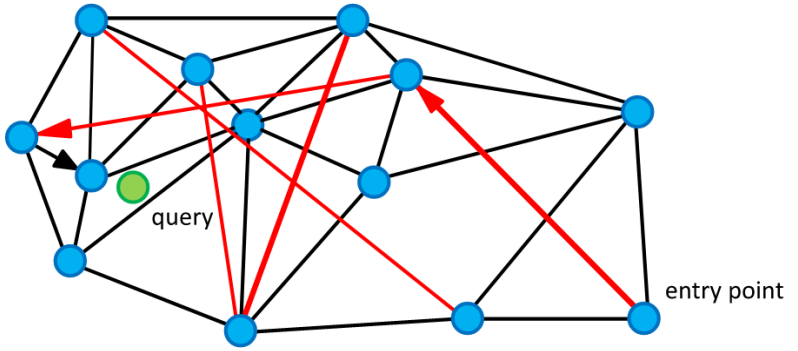
[1]Shanghai Jiao Tong University, [2]Peking University

Background

- Memory wall
  - The mismatch between CPU speed and memory speed
- Approximate Nearest Neighbor Search (ANNS)
  - Trading a marginal reduction in recall for reduced search latency
  - Foundational for IR, recommendation systems, RAG
- ANNS faces the "memory wall" bottleneck
  - Extremely low compute-to-memory access ratio
  - Negligible temporal locality in search vectors



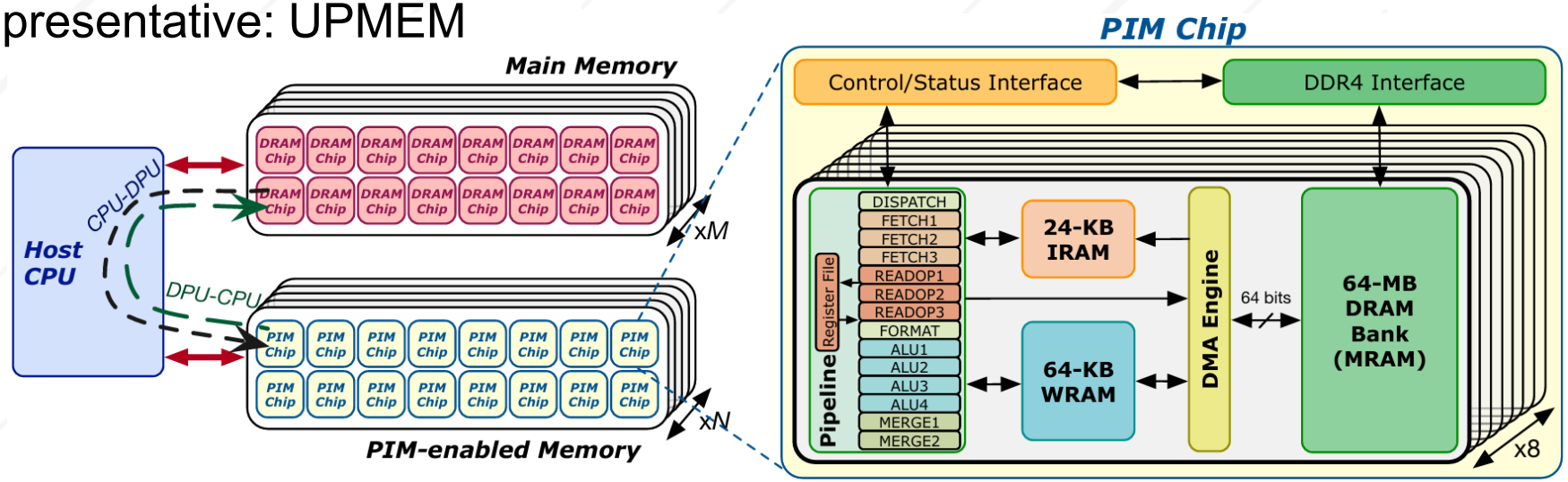
*Reproduced from Hennessy J L, Patterson D A. Computer architecture: a quantitative approach[M]. 6th ed. Cambridge, MA: Morgan Kaufmann, 2019: 80



*Reproduced from Vasnetsov A. Filterable approximate nearest neighbor search[EB/OL]. (2019-11-24) [2026-04-22]. https://blog.vasnetsov.com/posts/categorical-hnsw/.

Background

- Processing-in-Memory
  - Enable computations within the device's memory
  - Typical structure
    - A powerful host CPU and a set of PIM modules with limited the computational capability
    - PIM offers high intra-bandwidth but relatively low inter module bandwidth
  - Representative: UPMEM



*Reproduced from Gómez-Luna J, et al. Benchmarking memory-centric computing systems: Analysis of real processing-in-memory hardware[C]//2021 12th International green and sustainable computing conference (IGSC). IEEE, 2021: 1-7.

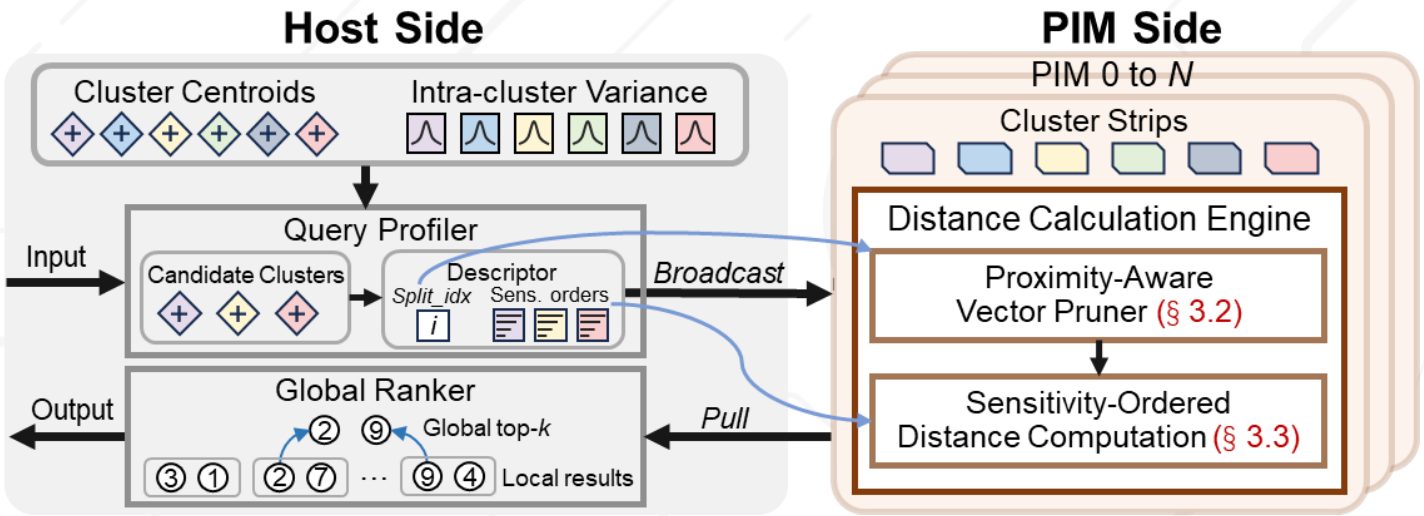


Challenge

- Mismatch between computational capability and memory bandwidth
  - PIM modules
    - High intra-module bandwidth
    - Weak on-die cores
    - Data cannot be shared across modules
  - ANNS
    - Entails a vast number of distance calculations among vectors
- Balance efficiency and accuracy
  - Graph-based search: substantial inter-PIM communication overhead
  - Cluster-based search: exhaustive distance computation remains the bottleneck
  - Quantization method: alleviates the arithmetic pressure but leads to degraded recall

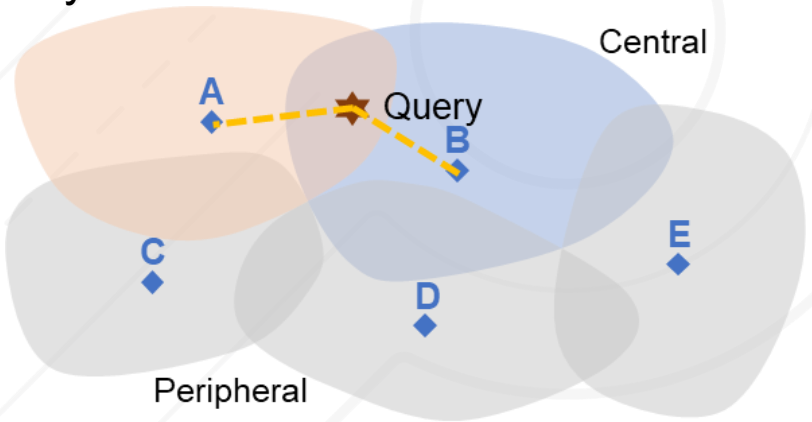
Design

- System Architecture
  - Main components: Query profiler → Distance calculation engine → Global ranker
  - Query descriptor
    - *split_idx*: guides vector pruning
    - Sensitivity order: guides early termination
  - PIM-side execution
    - Proximity-aware vector pruning
    - Sensitivity-ordered computation
  - Cache-efficient global ranking
- Data layout
  - Cluster strip sharding
  - Broadcast based query



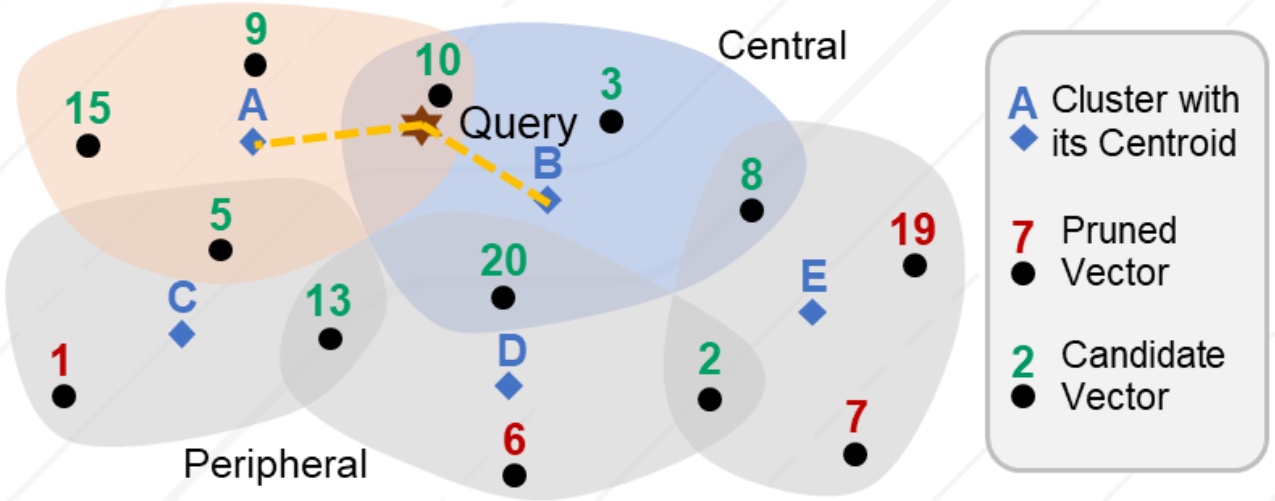
Design

- Proximity-Aware Vector Pruner
  - Distant vectors in selected clusters are unlikely in top-*k* yet still computed
  - Dual-affiliation vector clustering (offline)
    - 1) Perform *k*-means to assign primary centroid
    - 2) Affiliates each vector with its second-nearest centroid
  - Central-peripheral cluster partitioning (online)
    - 1) Select top-*n* nearest clusters by centroid-query distance
    - 2) Compute d_{bound} as weighted average of centroid-query distances
    - 3) *split_idx* is last index *i* where $d_i <= d_{bound}$
  - Central clusters: all vectors retained
  - Peripheral clusters: subject to pruning



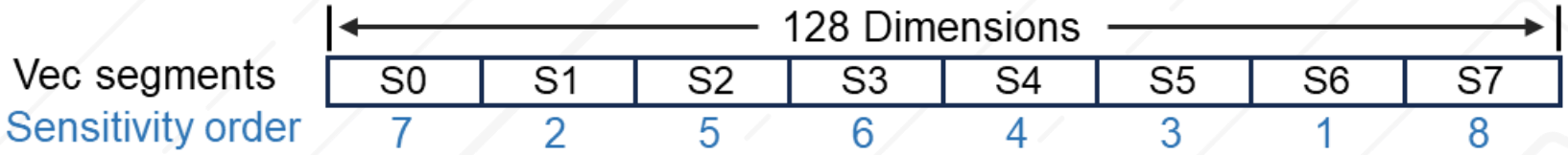
Design

- Proximity-Aware Vector Pruner
  - Intersection-based vector pruning
    - Discard: either affiliated cluster falls outside selected set → vector is distant
    - Retain: both affiliated clusters are within selected set → vector is spatially close
  - Yield compact candidate vector set



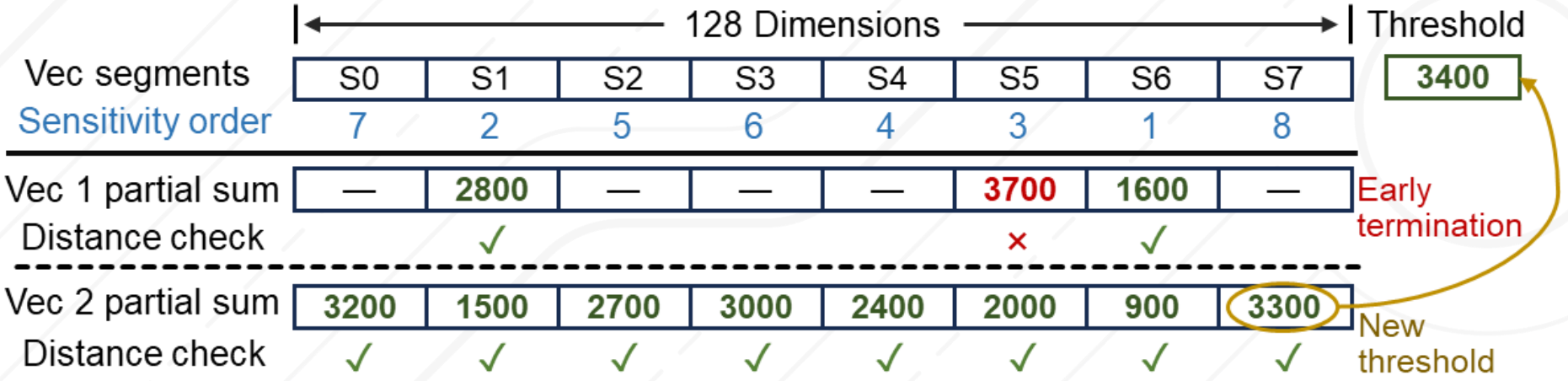
Design

- Sensitivity-Ordered Distance Computation
  - Reducing unnecessary computations is critical
    - Limited compute resources on the PIM side
    - Conventional sequential partial sum accumulation is inefficient
  - Segment sensitivity order
    - Partition vector into fixed-sized, non-overlapping segments
    - Host scores each segment by combining:
      - *Intra-cluster* variance (offline)
      - *query-centroid* variance (online)
    - Segments with higher combined variance are considered more sensitive
      - Higher sensitivity indicates greater discriminative power



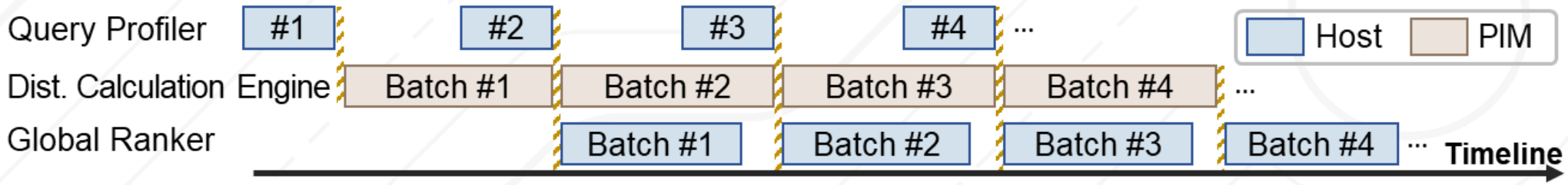
Design

- Sensitivity-Ordered Distance Computation
  - Distance computation with adaptive early termination
    - After each segment is accumulated, the partial sum is compared against the threshold
      - If exceeded, the vector is discarded
    - Processing high-impact segments first increases the likelihood of early termination
      - Vectors that finish below the threshold dynamically lower the threshold



Design

- Stall-Free Execution Pipeline
  - Overlap host preparation, PIM computation, and host ranking across adjacent batches
  - Eliminate inter-batch idle time and sustain high utilization
    - Stage 1: Host-side preparation
    - Stage 2: PIM-side execution
    - Stage 3: Host-side ranking





Evaluation

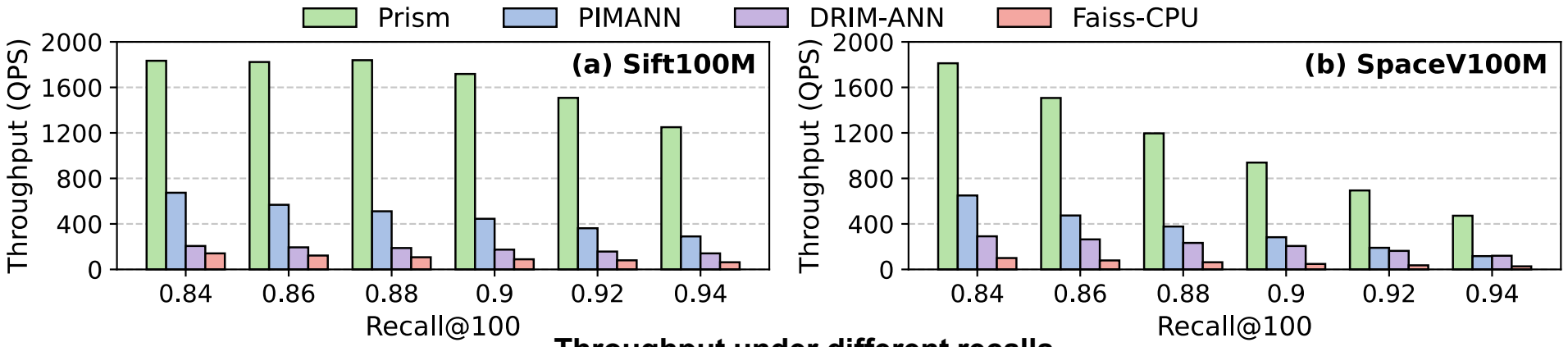
- Experiment Setup
  - Datasets
    - Sift100M, 100 million 128-dimension uint8 vectors
    - SpaceV100M, 100 million 100-dimension int8 vectors
  - Compared systems
    - Faiss-CPU: widely adopted ANNS library from Meta
    - DRIM-ANN: leverage batch processing and Product Quantization (PQ)
    - PIMANN: employ a fine-grained, per-DPU scheduling strategy, built upon PQ
  - Metrics
    - Recall@100: the overlap between result set and the ground-truth top-100 items
      - PQ-based methods require returning more results to the host to meet the target recall
    - Throughput (QPS) and latency at equivalent Recall@100

Evaluation

- Experiment Setup
  - Testbed
    - Server: 2× Intel Xeon Silver 4216 CPUs, 256 GB DDR4-3200 DRAM
    - Memory: 20 UPMEM DIMMs (2560 PIM modules total)
    - DPU per PIM module: 350 MHz, 24 hardware threads
  - Workload Setup
    - The number of clusters to 4096
    - Batch size to 128
    - For Prism:
      - Each PIM module produce 2 local results per query
      - Partition vectors into 8 segments for Sift100M and 10 segments for SpaceV100M
      - For 8-bit integer datasets (e.g., uint8/int8), precompute a square table and use table lookup instead of multiplication.

Evaluation

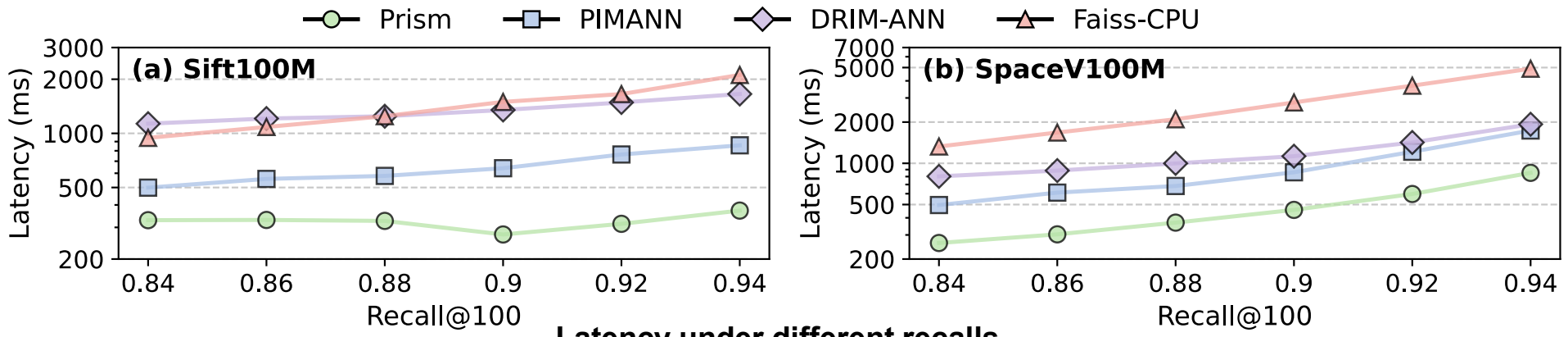
- QPS Under Different Recall
  - vs. Faiss-CPU (13.0 - 19.8×)
    - Effectively leverage high internal PIM bandwidth
  - vs. DRIM-ANN (3.9 - 9.9×)
    - Stall-free execution pipeline + Reduced computation
  - vs. PIMANN (2.7 - 4.3×)
    - Eliminate per-DPU scheduling overhead + Remove PIM-side result merging pressure



Throughput under different recalls.

Evaluation

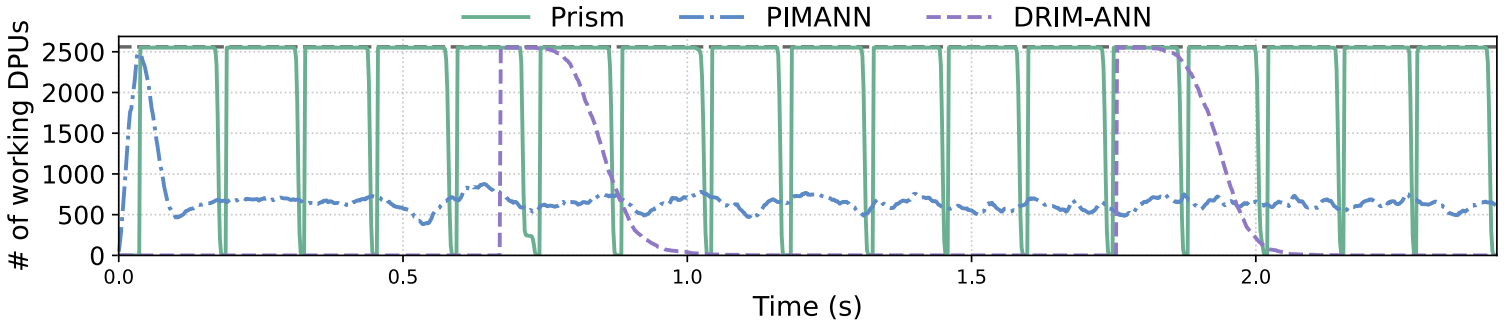
- Latency Under Different Recall
  - vs. Faiss-CPU (65 - 84% lower)
    - Massive PIM parallelism for efficient vector access and concurrent distance computation
  - vs. DRIM-ANN (56 - 80% lower)
    - Reduced on-die compute pressure + Highly load-balanced data placement
  - vs. PIMANN (34 - 58% lower)
    - No low-bandwidth control interface bottleneck + Less host-side result transfer



Latency under different recalls.

Evaluation

- Working DPU Analysis
  - PIMANN: working DPUs drop quickly
    - Polling-based scheduler causes dispatch latency and load imbalance
    - Popular slices overload, most DPUs idle
  - DRIM-ANN: gradual intra-batch decline + Inter-batch idleness
    - Cluster-based layout + No host-PIM pipeline leave DPUs underutilized
  - Prism: working DPUs remain consistently high
    - Uniform sharding + Broadcast dispatch eliminate imbalance
    - Stall-free pipeline removes inter-batch gaps



Number of working DPUs across systems on SpaceV100M (Recall@100 = 0.90).

Conclusion

- Prism, a PIM-based ANNS engine that reduces arithmetic while preserving recall
  - Proximity-Aware Vector Pruner
    - Filters distant vectors
  - Sensitivity-Ordered Distance Computation
    - Evaluate segments in descending significance
    - Early termination once vector exclusion is certain
  - Stall-Free Host-PIM Pipeline
    - Overlaps host and PIM execution to eliminate bubbles
  - High QPS and low latency over state-of-the-art PIM-based and conventional ANNS systems

Weihan Kong
weihankong@sjtu.edu.cn